INAP
INSTITUTO NACIONAL DE ADMINISTRACIÓN PÚBLICA

Curso online: **Programación orientada a objetos: Java avanzado**

Ejercicio 4

Autores

Jorge Molinero Muñoz

Miguel Sallent Sánchez

ÍNDICE

1. Objetivo..... 3
2. Primera parte: streams y expresiones lambda..... 3
3. Segunda parte: pruebas unitarias..... 5
4. Entrega..... 5

1. Objetivo

El objetivo de este ejercicio es poner en práctica el uso de streams y de expresiones lambda.

2. Primera parte: streams y expresiones lambda

Se facilitan un conjunto de clases que modelan los expedientes de un conjunto de ciudadanos.

- Ciudadano: tiene una lista de expedientes asociados
- Expediente: tiene dos listas asociadas: una de entradas y otra de salidas
- EntradaRegistro
- SalidaRegistro

Estas clases están disponibles en la plataforma (el alumno no tiene que implementarlas).

El alumno debe implementar la clase Filtros, de la que se facilita el esqueleto. Esta clase tiene unos métodos estáticos que, para una lista de ciudadanos de entrada, permiten extraer cierta información que se detalla más adelante.

Aunque el ejercicio se podría realizar sin utilizar streams, es **obligatorio** implementar los métodos utilizando streams y expresiones lambda. Todos los métodos se pueden (y deben) implementar en una sola sentencia.

Todos los métodos tienen como entrada una lista de objetos Ciudadano.

Se facilita también la clase PruebaFiltros. Esta clase crea un conjunto de datos de prueba e invoca a los métodos de la clase Filtros para probar su correcto funcionamiento.

A continuación se detallan los métodos de la clase Filtros que debe implementar el alumno:

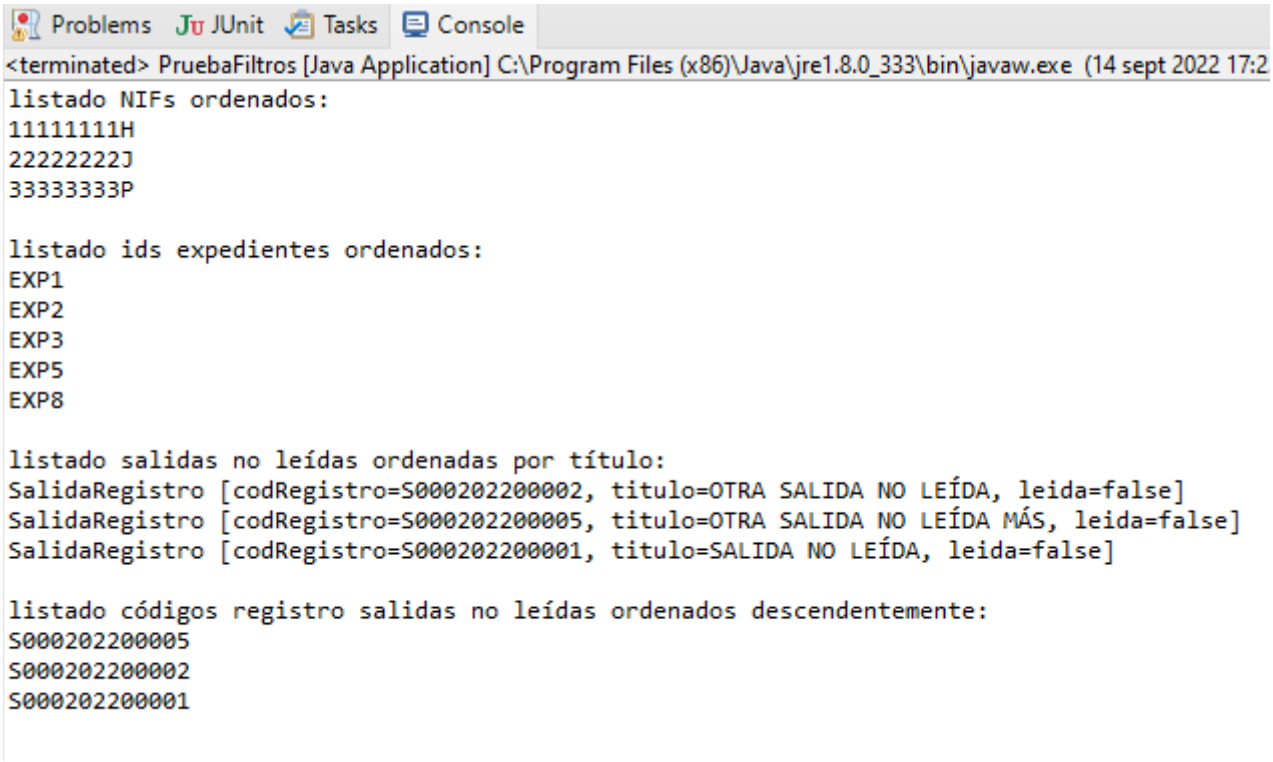
- obtenerNifsCiudadanos
  - Debe devolver una lista con los NIF de los ciudadanos, ordenados ascendentemente (orden alfabético)
  - Recuerda utilizar stream
  - Para convertir un stream de ciudadanos en un stream de NIF podemos usar el método map (relación uno a uno)
  - Para convertir un stream en una lista, podemos usar el método collect, pasando como argumento “Collectors.toList()”
- obtenerIdsExpedientes
  - Debe devolver una lista con los id de expediente de todos los expedientes de la lista de ciudadanos, ordenados ascendentemente.

- Podemos usar el método flatMap para pasar de un stream de ciudadanos a un stream de expedientes (relación uno a varios)

- obtenerSalidasNoLeidas
  - Debe devolver una lista con todas las salidas no leídas, ordenadas por título
  - Usaremos flatMap dos veces, primero para obtener los expedientes y luego para obtener las salidas
  - Se puede usar el método filter para filtrar las salidas no leídas
  - Para ordenar por título habrá que utilizar un Comparator como argumento del método sorted. Recuerda que el método compare de Comparator tiene dos argumentos, por tanto la expresión lambda tendrá dos argumentos.

- obtenerCodigosRegistroSalidasNoLeidas
  - Debe devolver una lista con los códigos de registro de todas las salidas no leídas, ordenados descendentemente
  - Similar al método anterior, pero después del filtro aplicaremos map para convertir de stream de objetos Salida a stream de String con los códigos de registro
  - Para obtener el orden descendente, al método sorted podemos pasarle como argumento "Comparator.reverseOrder()"

Ejecución de la primera parte



3. Segunda parte: pruebas unitarias

En esta parte del ejercicio el alumno deberá implementar una clase FiltrosTest, que será una clase de pruebas Junit.

Dentro de esta clase se debe implementar al menos una prueba unitaria por cada uno de los cuatro métodos de la primera parte.

Podemos implementar un método con la anotación @BeforeEach, y dentro de este método construir la lista de ciudadanos de prueba, que será un atributo de la clase. De esta forma usaremos la misma lista de ciudadanos para todos los test. Otra opción sería que cada prueba unitaria construyera su lista de ciudadanos particular (queda a elección del alumno).

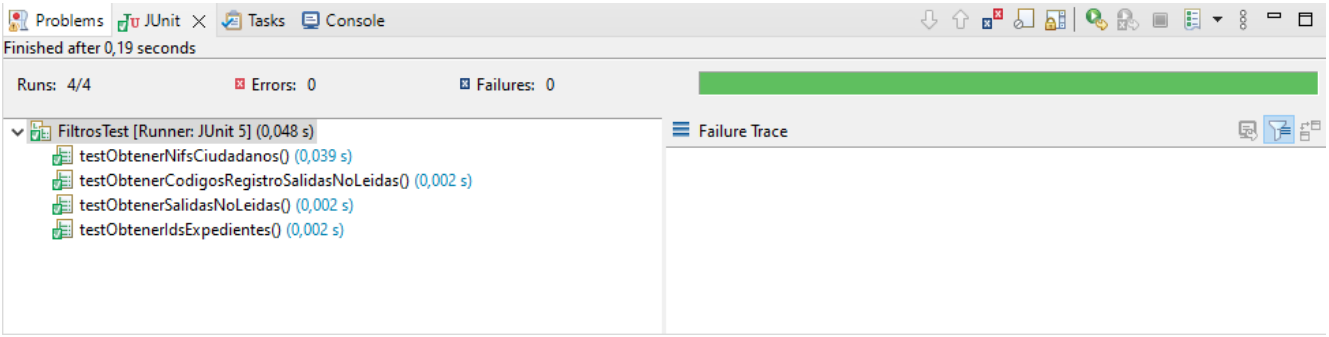
En cada prueba se deben hacer al menos las siguientes comprobaciones (assertions):

- Que el número de elementos en la lista devuelta por el método es el esperado.
- Que cada uno de los elementos en la lista devuelta es el esperado. Esto se podría hacer por ejemplo con un iterator e ir comprobando que cada elemento iterator.next() es el esperado.

Respecto a los datos de prueba, se pueden usar los mismos que los de la primera parte. O bien hacer una nueva lista de prueba, en este caso la lista debería tener:

- Al menos dos ciudadanos
- Al menos uno de los ciudadanos debe tener más de un expediente
- Al menos uno de los expedientes debe tener más de una entrada y más de una salida

Ejecución de la segunda parte



4. Entrega

El alumno entregará el proyecto Eclipse con todos los ficheros fuente del ejercicio, en un archivo zip o similar.